# Section 17 : Smart Pointers

## Information

- Title: `Smart Pointers`
- Authors: `Etienne KOA`

## Aperçu

### Pointeurs intelligents

- Problèmes avec les pointeurs bruts
- Que sont les pointeurs intelligents ?
- Concept de propriété et RAII
- Pointeurs intelligents `C++`
  - Pointeurs uniques(`unique_ptr`)
  - Pointeurs partagés(`shared_ptr`)
  - Pointeurs faibles (`weak_ptr`)
- Supprimeurs personnalisés

## Quelques problèmes avec les pointeurs bruts

### Problèmes avec les pointeurs bruts

- `C++` offre une flexibilité absolue dans la gestion de la mémoire
  - Allocation
  - Désallocation
  - Gestion à vie
- Quelques problèmes potentiellement graves
  - Pointeurs non initialisés (sauvages)
  - Fuites de mémoire
  - Pointeurs pendants
  - Pas d'exception sans danger
- Propriété ?
  - À qui appartient le pointeur ?
  - Quand faut-il supprimer un pointeur ?

## Qu'est-ce qu'un pointeur intelligent ? Propriété et RAII

Quels sont-ils ?

- Objets
- Ne peut pointer que vers la mémoire allouée au tas
- Appeler automatiquement la suppression lorsque vous n'en avez plus besoin
- Adhérer aux principes RAII
- Pointeurs intelligents `C++`
  - Pointeurs uniques(`unique_ptr`)
  - Pointeurs partagés(`shared_ptr`)

  - Pointeurs faibles (`weak_ptr`)
  - Pointeurs automatiques (`auto_ptr`) Obsolète- nous n'en discuterons pas

```
#include <memory>
```

- Défini par les modèles de classe
  - Wrapper autour d'un pointeur brut
  - Opérateurs surchargés
    * Sélection de déférence `(*)`
    * Sélection des membres `(->)`
    * L'athmétique du pointeur n'est pas prise en charge `(++, --, etc.)`
- Peut avoir des suppresseurs personnalisés

**Un exemple simple**

```
{
   std::smart_pointer<Some_Class> ptr = . . .

   ptr->method();
   cout << (*ptr) << endl;
}

// ptr will be destroyed automatically when
// no longer needed
```

Figure 1: Smart_Pointers

**RAII - L'acquisition de ressources est une initialisation**

- Idiome ou modèle commun utilisé dans la conception de logiciels basé sur la durée de vie des objets conteneurs.
- Les objets `RAII` sont alloués sur la pile
- Acquisition de ressources
  - Ouvrir un fichier
  - Allouer de la mémoire
  - Acquérir une serrure
- Est l'initialisation
  - La ressource est acquise chez un constructeur
- Abandon des ressources
  - Fermer un fichier
  - Désallouer la mémoire
  - Libérez le bloc

## Pointeurs uniques

`unique_ptr`

- Pointeur intelligent simple - très efficace !
- `unique_ptr`
  - Pointe vers un objet pour taper « T » sur le tas
  - Il est unique - il ne peut y avoir qu'un seul `unique_ptr<T>` pointant vers l'objet sur le tas
  - Possède ce vers quoi il pointe
  - Ne peut pas être attribué ou copié
  - PEUT être déplacé
  - Lorsque le pointeur est détruit, ce qu'il pointe est automatiquement détruit

**`unique_ptr` - Création, initialisation et utilisation**

```
{
   std::unique_ptr<int> p1 {new int {100} };

   std::cout << *p1 << std::endl;    // 100

   *p1 = 200;

   std::cout << *p1 << std::endl;    // 200

}// automatically deleted
```

Figure 2: Smart_Pointers

**`unique_ptr` - Quelques autres méthodes utiles**

**`unique_ptr` - classes définies par l'utilisateur**

**`unique_ptr` - vecteurs et déplacement**

**`unique_ptr` - make_unique (C++14)**

## Pointeurs partagés

`shared_ptr`

- Fournit la propriété partagée des objets du tas
- `shared_ptr<T>`
  - Pointe vers un objet de type `T` sur le tas

```
{
   std::unique_ptr<int> p1 {new int {100} };

   std::cout << p1.get() << std::endl;  // 0x564388

   p1.reset(); // p1 is now nullptr

   if (p1)
     std::cout << *p1 << std::endl;     // won't execute
}// automatically deleted
```

Figure 3: Smart_Pointers

```
{
   std::unique_ptr<Account> p1 {new Account{"Larry"}};
   std::cout << *p1 << std::endl;  // display account

   p1->deposit(1000);
   p1->withdraw(500);


}// automatically deleted
```

Figure 4: Smart_Pointers

```
{
   std::vector<std::unique_ptr<int>> vec;

   std::unique_ptr<int> ptr {new int{100}};

   vec.push_back(ptr);   // Error - copy not allowed

   vec.push_back(std::move(ptr));

}// automatically deleted
```

Figure 5: Smart_Pointers

```
{
   std::unique_ptr<int> p1 = make_unique<int>(100);

   std::unique_ptr<Account> p2  = make_unique<Account>("Curly", 5000);

   auto p3 = make_unique<Player>("Hero", 100, 100);

}// automatically deleted
```

Figure 6: Smart_Pointers

- C'est unique - il peut y avoir plusieurs `shared_ptr` pointant vers le même objet sur le tas
- Établit une relation de propriété partagée
- PEUT être attribué et copié
- PEUT être déplacé
- Ne prend pas en charge la gestion des tableaux par défaut
- Lorsque le nombre d'utilisations est nul, l'objet géré sur le tas est détruit

**`shared_ptr` - Création, initialisation et utilisation**

```
{
   std::shared_ptr<int> p1 {new int {100} };

   std::cout << *p1 << std::endl;    // 100

   *p1 = 200;

   std::cout << *p1 << std::endl;    // 200

}// automatically deleted
```

Figure 7: Smart_Pointers

**`shared_ptr` - Quelques autres méthodes utiles**

**`shared_ptr` - classes définies par l'utilisateur**

**`shared_ptr` - vecteurs et déplacement**

**`shared_ptr` - make_shared (C++14)**

```
{
   // use_count - the number of shared_ptr objects managing the heap object
   std::shared_ptr<int> p1 {new int {100} };
   std::cout << p1.use_count () << std::endl;        // 1

   std::shared_ptr<int> p2 { p1 };                   // shared ownwership
   std::cout << p1.use_count () << std::endl;        // 2

   p1.reset();      // decrement the use_count; p1 is nulled out
   std::cout << p1.use_count() << std::endl;         // 0
   std::cout << p2.use_count() << std::endl;         // 1
}  // automatically deleted
```

Figure 8: Smart_Pointers

```
{
   std::shared_ptr<Account> p1 {new Account{"Larry"}};
   std::cout << *p1 << std::endl;  // display account

   p1->deposit(1000);
   p1->withdraw(500);

}// automatically deleted
```

Figure 9: Smart_Pointers

```
{
   std::vector<std::shared_ptr<int>> vec;

   std::shared_ptr<int> ptr {new int{100}};

   vec.push_back(ptr);   // OK - copy IS allowed

   std:: cout << ptr.use_count() << std::endl;    // 2
}// automatically deleted
```

Figure 10: Smart_Pointers

```
{
   std::shared_ptr<int> p1 = std::make_shared<int>(100); // use_count: 1
   std::shared_ptr<int> p2 { p1 };                  // use_count : 2
   std::shared_ptr<int> p3;
   p3 = p1;                                         // use_count : 3

}// automatically deleted
```

Figure 11: Smart_Pointers

## Pointeurs faibles

`weak_ptr`

- Fournit une référence « faible » non propriétaire
- `weak_ptr`
  - Pointe vers un objet de type `T` sur le tas
  - Ne participe pas à la relation de propriété
  - Toujours créé à partir d'un `shared_ptr`
  - N'incrémente ni ne décrémente le nombre d'utilisations de référence
  - Utilisé pour empêcher les cycles de référence forts qui pourraient empêcher la suppression des objets

`weak_ptr` - Référence circulaire ou cyclique

- A fait référence à B
- B fait référence à A
- Une forte propriété partagée empêche la désallocation du tas
- Solution - rendre l'un des pointeurs non propriétaire ou « faible »
- Désormais, le stockage en tas est désalloué correctement

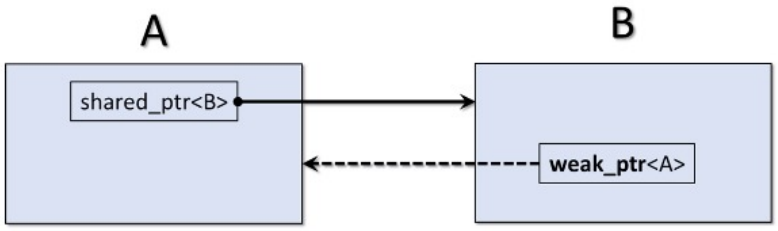


Figure 12: Smart_Pointers

## Supprimeurs personnalisés

- Parfois, lorsque nous détruisons un pointeur intelligent, nous avons besoin de plus que simplement détruire l'objet sur le tas
- Ce sont des cas d'utilisation spéciaux
- Les pointeurs intelligents `C++` vous permettent de fournir des suppresseurs personnalisés
- De nombreuses façons d'y parvenir
  - Fonctions
  - Lambda
  - Autres...

### Supprimeurs personnalisés - fonction

```
void my_deleter(Some_Class *raw_pointer) {
   // your custom deleter code
   delete raw_pointer;
}


shared_ptr<Some_Class> ptr { new Some_class{}, my_deleter };


void my_deleter(Test *ptr) {
   cout << "In my custom deleter" << end;
   delete ptr;
}


shared_ptr<Test> ptr { new Test{}, my_deleter };
```

### Supprimeurs personnalisés - lambda

```
shared_ptr<Test> ptr (new Test{100}, [] (Test *ptr)  {
      cout << "\tUsing my custom deleter" << endl;
      delete ptr;
});
```

Figure 13: Smart_Pointers